

HelixSpecifier

HelixSpecifier

**İşin gerektirdiği ölçüde kendi törenini ölçeklendiren
spesifikasyon odaklı geliştirme.**

Özet

HelixSpecifier, üç geliştirme metodolojisini —SpecKit'in spesifikasyon odaklı iş akışı, Superpowers'ın TDD disiplini ve GSD'nin kilometre taşı yaşam döngüsü— uyarlanabilir tek bir akışta birleştiren bir Go motorudur. Her görevi efor düzeyine göre sınıflandırır ve süreç miktarını buna göre artırır ya da azaltır.

Kısa açıklama

HelixSpecifier, AI ajanları için spesifikasyon odaklı geliştirme füzyon motorudur. SpecKit, Superpowers ve GSD'yi bir araya getirir, işleri efor düzeyine göre sınıflandırır, tartışma destekli spesifikasyon aşamaları yürütür, minimum test-kod oranını zorunlu kılar ve tamamlanan her akıştan öğrenir.

Uzun açıklama

HelixSpecifier, Go'da (modül `digital.vasic.helixspecifier`) yazılmış, spesifikasyon odaklı geliştirme (SDD) füzyon motorudur ve HelixAgent AI topluluğunun bir bileşeni olarak tasarlanmıştır. Normalde üç ayrı araçta —ve üç ayrı zihniyette— yer alan üç geliştirme pratiğini tek bir uyarlanabilir iş akışında birleştirir: SpecKit'in yedi aşamalı SDD süreci (Constitution, Spesifikasyon, Açıklama, Planlama, Görevler, Analiz, Uygulama), Superpowers'ın paralel alt ajan yürütmeli test odaklı disiplini ve GSD'nin kilometre taşı yaşam döngüsü yönetimi. Her bir sütun kendi uzmanlık alanında çalışmaya devam eder; onları el yapımı bir boru hattı yerine tek bir tutarlı akış olarak işleten, motordur.

Merkezindeki fikir *uyarlanabilir törendir*: Motor, gelen işleri efor düzeyine göre sınıflandırır ve sürecin miktarını buna göre ölçeklendirir. Böylece tek satırlık bir düzeltme, büyük bir özellikte aynı ağırlıktaki bir ritüelden geçirilmez; büyük bir özellik de bir yazım hatası gibi gevşek bir titizlikle geçirilmez. Bu omurgaya dayanan on güçlü özellik şunlardır: sınırlı eşzamanlılıkla paralel görev yürütme, zorunlu kuralları otomatik olarak uygulayan makine tarafından okunabilir “Constitution as Code”, minimum test-uygulama oranını izleyen ve zorunlu kılan “Nyquist TDD”, spesifikasyonun iyileştirilmesi için çok turlu ve çok ajanlı tartışma, uyarlanabilir beceri yeterliliği öğrenimi, eski kodların analizi, geçmiş desenlerden çıkarılan tahmini spesifikasyon, projeler arası bilgi aktarımı, uçuş sırasında yeniden ayarlanan çalışma zamanı töreni ve anlamsal arama özelliğine sahip kalıcı spesifikasyon belleği.

Go modülü olarak —go get veya yerel bir replace yönergesi aracılığıyla— kasıtlı olarak küçük bir motor API’un arkasında tüketilir: Üç sütun ile bir tören ölçekleyici ve spesifikasyon belleği kaydedilir, işin eforu sınıflandırılır, ardından tam akış yürütülür ve kalite puanlı bir sonuç alınır. Yüzeyi basittir; arkasındaki orkestrasyon ise öyle değildir. Helix ailesinin geri kalanı gibi, sahte veriler yerine gerçek kodla çalışan bir süreç içi meydan okuma yürütücüsüyle, blöf karşıtı bir doğrulama rejimi altında geliştirilmiştir.

Neden geliştirdik

Spesifikasyon odaklı geliştirme, titiz TDD ve kilometre taşı yönetimi genellikle üç ayrı uygulama ve üç ayrı araçla yürütülür. HelixSpecifier, bir AI ajanı (HelixAgent) için bu üçünü el ile birleştirmek yerine, tek bir tutarlı ve kendi kendini ölçeklendiren iş akışı olarak çalıştırabilmesi için geliştirildi.

İçerik

Neden bir oyun değiştirici?

Süreci işin boyutuna otomatik olarak uyarlıyor. Takımlar genellikle iki kötü uçtan birinde takılıp kalır: her şeye ağır bir seremoni uygulamak (güvenli ama yavaş, üstelik sessizce nefret uyandırır) ya da hiçbir şeye seremoni uygulamamak (hızlı, ta ki öyle olmamaya başlayana kadar). HelixSpecifier bu ödünleşmeyi ortadan kaldırarak seremoniyi her görevin sınıflandırılmış eforuna göre boyutlandırıyor ve iş ilerledikçe çalışma kendini ortaya koydukça süreci yeniden ayarlıyor. Daha önce pratikte mümkün olmayan yetenek, göreve göre kendini doğru boyutlandıran süreç

— üstelik buna ek olarak, tek bir kişinin ilk tahmininden ziyade çok turlu, çok ajanlı bir tartışmayla desteklenen ve pozisyon puanlamasıyla alınan spesifikasyon kararları.

Yenilikçi olan ne?

- **Uyarlanabilir seremoni** — süreç seviyesi, önceden sabitlenmek yerine gerçek zamanlı kalite metrikleriyle yönlendirilir ve çalışma sırasında ayarlanır.
- **Nyquist TDD** — uygulama oranına göre bir test kapısı (en az 2x), Nyquist örnekleme teoreminin mantığından ödünç alınmıştır: davranışı sadakatle yakalamak için örnekleme hızının çok üzerinde örnekleme gerekir, bu nedenle testler kapsadıkları kodu aşmalıdır.
- **Tartışma mimarisi** — spesifikasyonların çok turlu, çok ajanlı olarak iyileştirildiği, pozisyonların önerildiği, puanlandığı ve birleştirildiği bir süreç; tek bir görüşü, karşıt görüşlerle değiştirir.
- **Öngörücü spesifikasyon ve projeler arası aktarım** — motor, birikmiş akışları analiz ederek spesifikasyonları öngörür ve bir projeden elde edilen zor kazanılmış bilgileri bir sonrakine taşır.
- **Constitution Kodu** — zorunlu proje kurallarının makine tarafından okunabilir hale getirilip motor tarafından uygulanması; gözden geçirenlerin dikkatine bırakılmaz.

En büyük teknik zorluklar ve çözümlerimiz

- **Üç metodolojiyi birbirleriyle çatışmadan birleştirmek** — SpecKit, Superpowers ve GSD, her biri iş akışının kendilerine ait olduğunu varsayar. Ortak bir arayüzün arkasına her bir sütunu kaydeden ve tek bir paylaşılan akış yaşam döngüsü üzerinden çalıştıran bir füzyon motoruyla çözüldü; böylece üç ayrı süreç yerine tek bir bütünleşik süreç ortaya çıktı.
- **Belirli bir görevin gerçekte ne kadar süreceğine karar vermek** — tahmin çok yüksek olursa her şey yavaşlar; çok düşük olursa riskli işler kontrolsüz bir şekilde yayınlanır. İşin boyutunu belirleyen bir efor sınıflandırıcısı ve çalışma ilerledikçe süreci dinamik olarak ayarlayan bir seremoni ölçekleyiciyle çözüldü.
- **Her kararda bir insan denetçiye ihtiyaç duymadan spesifikasyon kalitesini yüksek tutmak** — tek seferlik spesifikasyonlar yerine, ajanların turlar boyunca rakip pozisyonları puanladığı tartışma destekli iyileştirmeye ve uygulamanın testlerini aşamayacağı Nyquist TDD oranlarının uygulanmasıyla çözüldü.

Teknoloji yığını

- **Go** — motorun çalışma zamanı yükü olmadan tek bir içe aktarılabilir ikili dosya olarak sunulmasını sağladığı için seçildi; sınırlı paralel görev dağıtımı ve çok ajanlı tartışma turlarını, iş parçacığı karmaşası olmadan yönetilebilir kılan eşzamanlılık modeli sayesinde.
- **logrus** — motor ve üç sütun boyunca yapılandırılmış günlük kaydı; böylece bir akışın kararları (sınıflandırma, seremoni değişiklikleri, tartışma sonuçları) sonradan okunabilir hale gelir.
- **SpecKit sütunu** — spesifikasyondan koda giden yedi aşamalı disiplinli süreç (Constitution → Spesifikasyon → Açıklama → Planlama → Görevler → Analiz → Uygulama), bir spesifikasyonun kod haline gelmesinin sağlam omurgasını oluşturur.
- **Superpowers sütunu** — paralel alt ajan yürütmesiyle TDD disiplini; uygulamanın dürüst ve hızlı kalmasını sağlayan test öncelikli titizlik ve dağıtım.
- **GSD sütunu** — kilometre taşları ve yaşam döngüsü yönetimi; akışa “tamamlanmış” hissi ve aşamalar boyunca ilerleme duygusu kazandırır.
- **Spesifikasyon bellek deposu** — geçmiş spesifikasyonların kalıcı, anlamsal olarak aranabilir bir dizini; öngörücü spesifikasyon ve projeler arası aktarımın sıfırdan başlamak yerine mümkün olmasını sağlayan temel yapı.

İçerik

Durum ve dürüstlük notları

- **Durum: beta.** HelixAgent’un bir Go modül bileşeni olarak tüketilmiştir.
- **Lisans: belirlenecek.** GitHub API aracılığıyla herhangi bir LİSANS tespit edilemedi — DOĞRULANMAMIŞ / beyan edilmemiş.
- Görünen ad “HelixSpecifier”, specifier deposuna karşılık gelmektedir.

Öncelik seviyesi: Helix-birincil.